

Recitation 10

Neural Networks II

How to calculate your grade?

$$Exercise_{grade} = 0.1 * Ex1 + 0.2 * Ex2 + 0.2 * Ex3 + 0.3 * Ex4 + 0.2 * Ex5$$

$$Exams_{grade} = \text{Average of top 5 grades}$$

$$Final_{before} = 0.23 * Exercise_{grade} + 0.75 * Exams_{grade} + attendance$$

$$Final_{after} = Norm(Final_{before}) + bonus$$

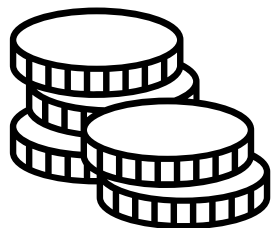
Agenda

1. Fine-tuning
2. LoRA
3. Generative Models

Fine-tuning

Challenges of training a model from scratch

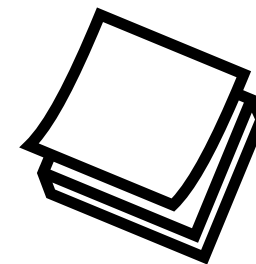
Expensive



Time-consuming



Hard to collect enough data



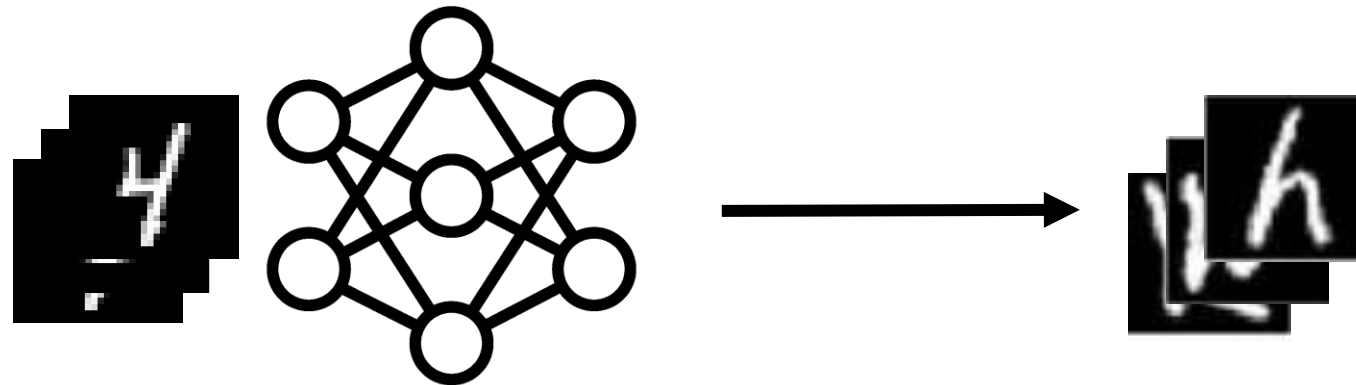
Idea:

Why not take a pretrained model and give it a small adjustment (“a little push”) for our task?

Fine-tuning

How Fine-Tuning Works

- Start with a model trained on a large dataset (ImageNet\MNIST).
- Keep the learned features (edges, shapes, textures).
- Train with a smaller dataset and fewer resources.



LoRA (Low-Rank Adaptation)

Key idea: Adapt big models with small changes.

- A method for fine-tuning large models efficiently.
- Instead of updating all weights, LoRA adds small trainable matrices (low-rank updates).
- The original model weights stay frozen - only a few extra parameters are learned (low rank matrices).
- Much faster and needs less memory than full fine-tuning.

Frozen

$$W \in \mathbb{R}^{n \times m}$$

Trained

$$A \in \mathbb{R}^{r \times m}$$

$$B \in \mathbb{R}^{n \times r}$$

$$r \ll n, m$$

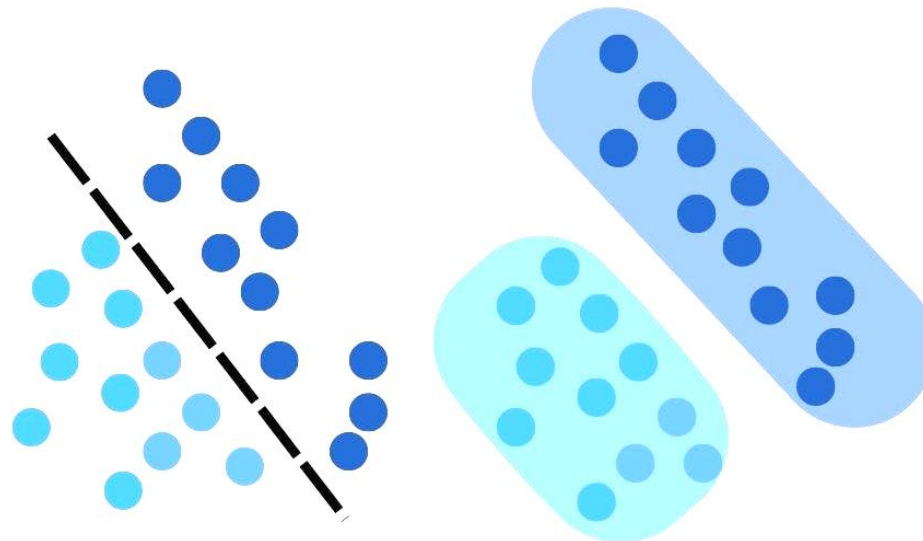
(small, 4 or 8)

$$h = (W + BA)x$$

Discriminative vs. Generative

Discriminative (classification/regression): Learn $p(y|x)$ - how to map inputs to labels/values.

Generative: Learn $p(x)$ or $p(x, y)$ - how to model or generate new data.



Discriminative

Generative

Loss Functions

Now, if our model's output is an image rather than a single number or class label, we need to use different types of loss functions.

Pixel-wise

$$L_1(I_t, I_o) = \|I_t - I_o\|_1 = \frac{1}{N} \sum_{x,y} |I_t[x, y] - I_o[x, y]|$$

gives sharper but possibly noisier results.

$$L_2(I_t, I_o) = \|I_t - I_o\|_2^2 = \frac{1}{N} \sum_{x,y} (I_t[x, y] - I_o[x, y])^2$$

gives blurry but globally accurate results.

Loss Functions

Perceptual Loss – Feature-based

Perceptual loss measures the difference between images in a deep feature space (pretrained VGG\ResNet), encouraging the generated image to look perceptually similar to the target rather than just matching pixel values.

$$L_{VGG}(I_t, I_o) = \|\phi_l(I_t) - \phi_l(I_o)\|_2^2$$



Original



l_2



Perceptual

Generative Models

Unsupervised learning

Generate **new** samples from the same distribution of the training data

A generative model needs to be:

- Sufficient: for every image x there must be z s.t. $G(z) = x$
- Compact: for every z , $G(z)$ should be a valid image in X

Many ways to address this problem

- Auto-regressive pixel generation
- VAE – variational autoencoders
- GAN – generative adversarial networks
- ViT – vision transformers
- Diffusion models

Generative Models

Unsupervised learning

Generate **new** samples from the same distribution of the training data

A generative model needs to be:

- Sufficient: for every image x there must be z s.t. $G(z) = x$
- Compact: for every z , $G(z)$ should be a valid image in X

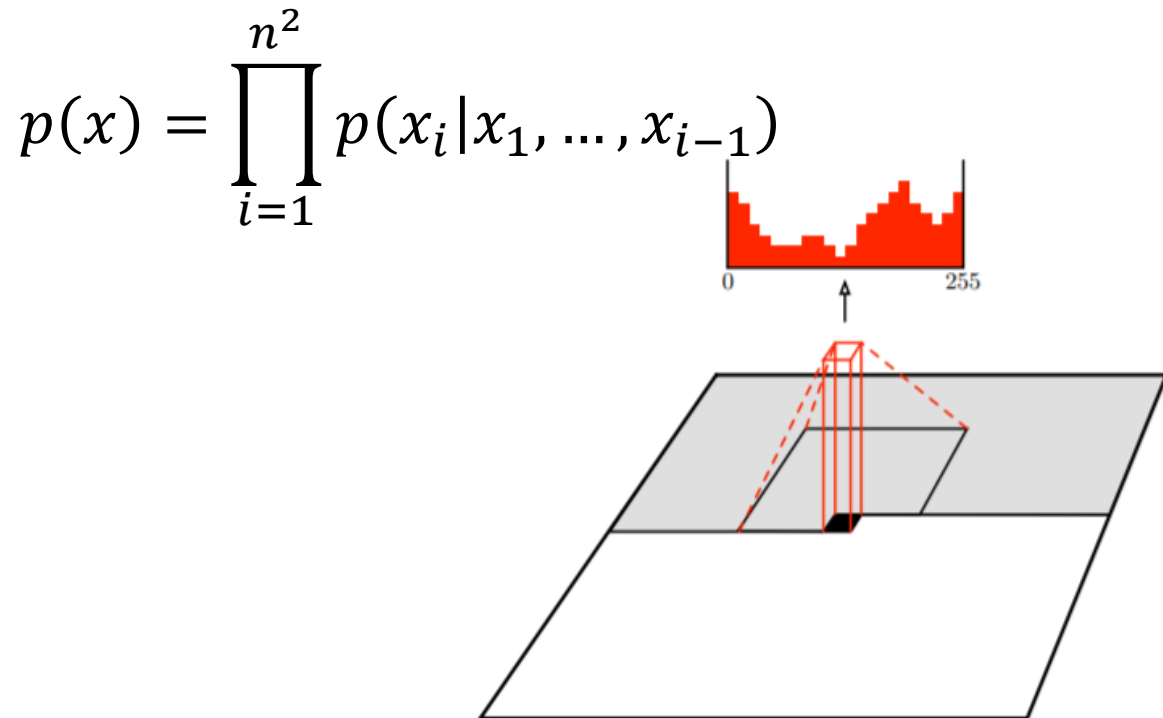
Many ways to address this problem

- **Auto-regressive pixel generation**
- VAE – variational autoencoders
- GAN – generative adversarial networks
- ViT – vision transformers
- Diffusion models

Auto-Regressive Pixel Generation

PixelCNN (van der Oord et al. 2016) – use all previously generated pixels to predict the next one, using auto-regressive convolutions

- Nice results but very slow – one pixel at a time



African elephant



Sorrel horse

Generative Models

Unsupervised learning

Generate **new** samples from the same distribution of the training data

A generative model needs to be:

- Sufficient: for every image x there must be z s.t. $G(z) = x$
- Compact: for every z , $G(z)$ should be a valid image in X

Many ways to address this problem

- Auto-regressive pixel generation
- **VAE – variational autoencoders**
- GAN – generative adversarial networks
- ViT – vision transformers
- Diffusion models

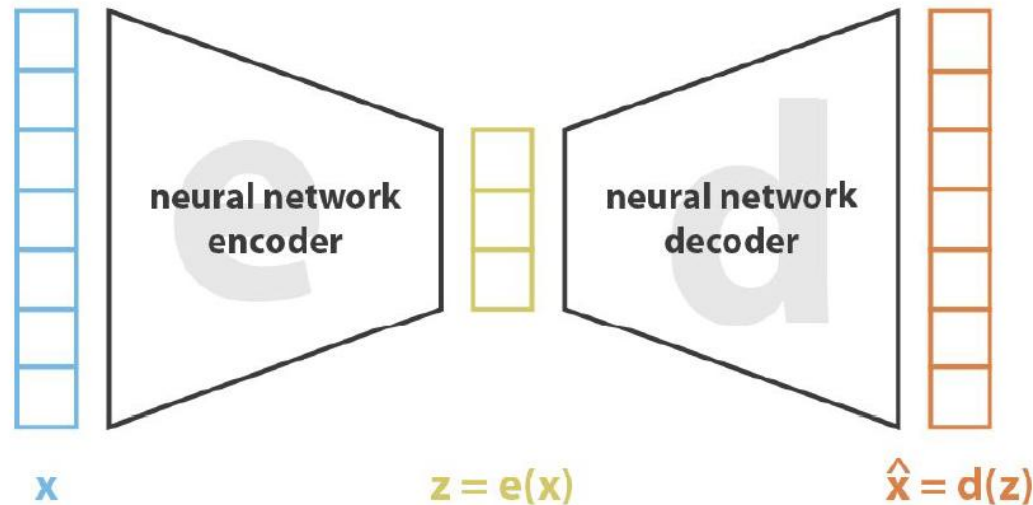
Autoencoders

Compress input to a low-dimensional latent vector

Unsupervised learning

Loss: $\|x - \hat{x}\|^2$

Latent vector can be the input to supervised classification network



VAE – Variational Autoencoders

Auto-Encoding Variational Bayes

Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

VAE – Variational Autoencoders

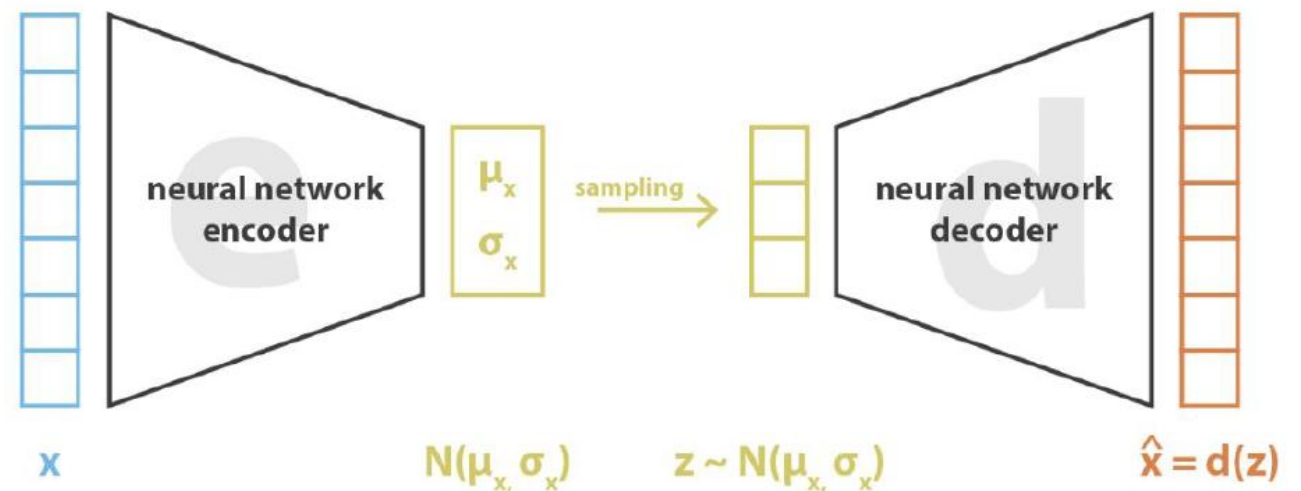
Sample latent vectors to create new images

Need to enforce some “arrangement” on the autoencoder’s latent space

Encourage latent vector to be normally distributed

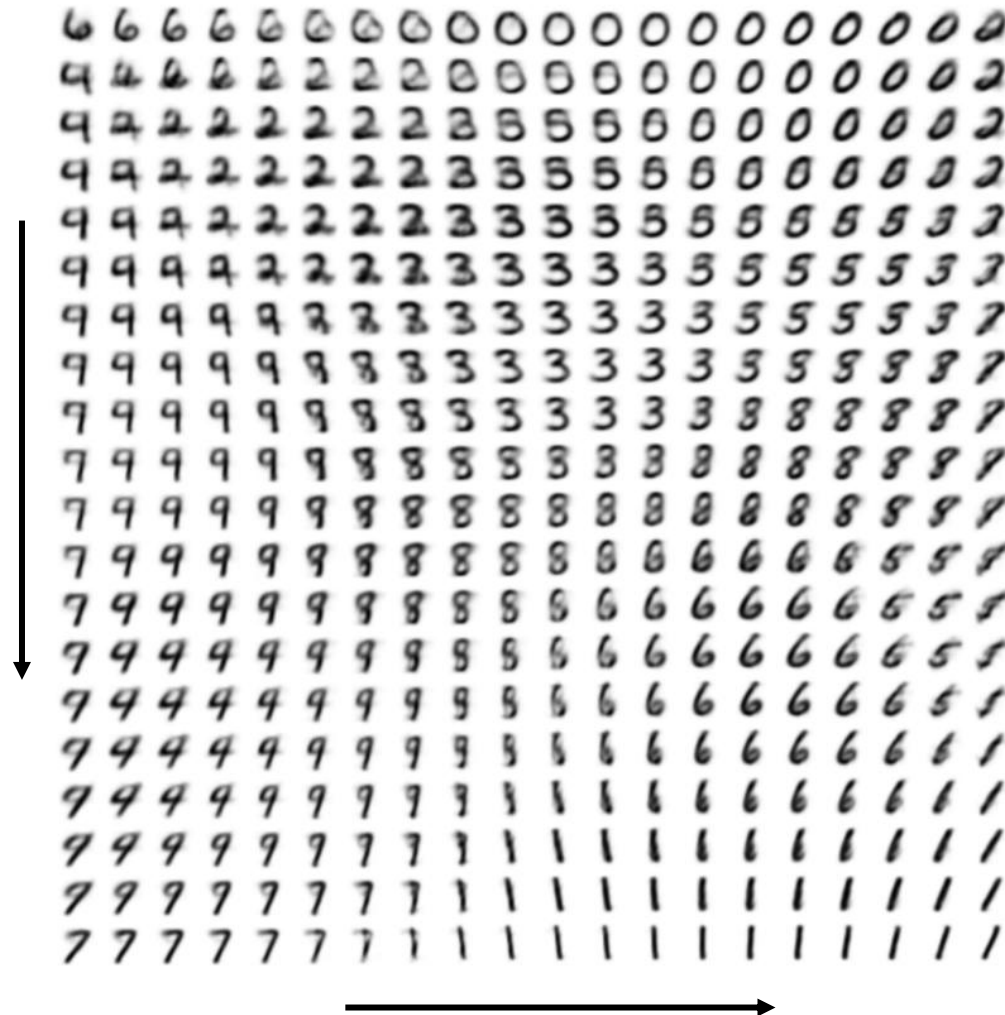
- Instead of learning latent vector, learn mean standard deviation and use them to sample from normal distribution

Loss would now be both reconstruction loss ($\|x - \hat{x}\|^2$) and regularization loss ($\text{KL}[N(\mu_x, \sigma_x) || N(0, I)]$)



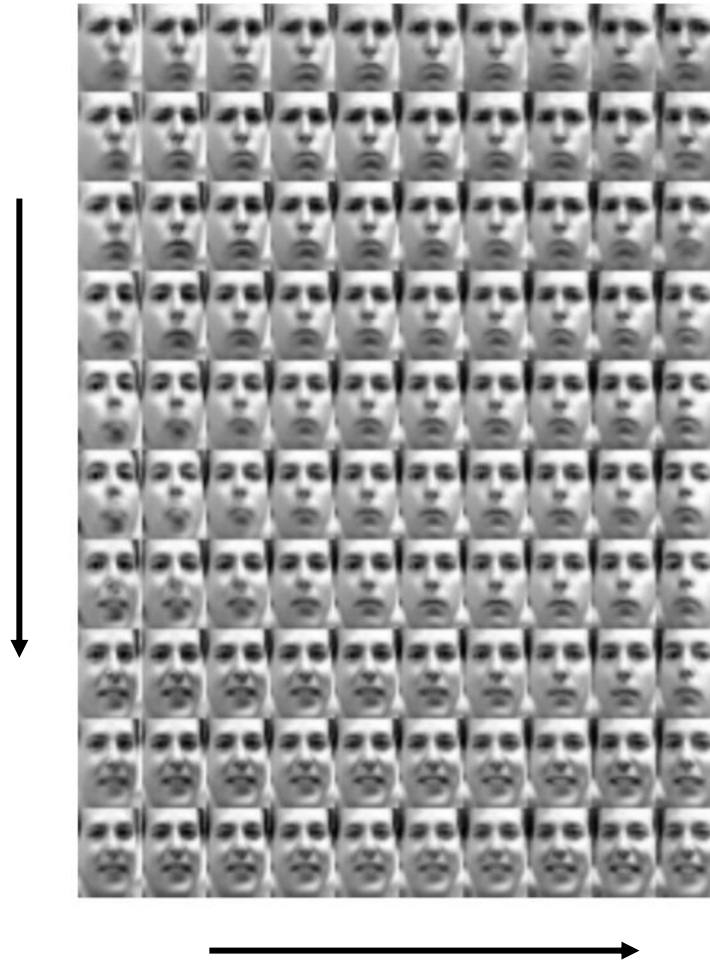
VAE – Variational Autoencoders

2D Z manifold



VAE – Variational Autoencoders

2D Z manifold



Generative Models

Unsupervised learning

Generate **new** samples from the same distribution of the training data

A generative model needs to be:

- Sufficient: for every image x there must be z s.t. $G(z) = x$
- Compact: for every z , $G(z)$ should be a valid image in X

Many ways to address this problem

- Auto-regressive pixel generation
- VAE – variational autoencoders
- **GAN – generative adversarial networks**
- ViT – vision transformers
- Diffusion models

GAN – Generative Adversarial Networks

Generative Adversarial Nets

**Ian J. Goodfellow^{*}, Jean Pouget-Abadie[†], Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair[‡], Aaron Courville, Yoshua Bengio[§]**

Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

GAN – Generative Adversarial Networks

Two player game:

- **Generator network G**: generate images that will look real enough to fool D
- **Discriminator network D**: distinguish between real images and images produced by G

Adversarial training – both networks trained together in a minimax game:

- Discriminator trained to maximize the difference between real and fake images
- Generator trained to minimize it

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] .$$

D's output for real images

D's output for images created by G

GAN – Generative Adversarial Networks

Two player game:

- **Generator network G:** generate images that will look real enough to fool D
- **Discriminator network D:** distinguish between real images and images produced by G

Adversarial training – both networks trained together in a minimax game:

- Discriminator trained to maximize the difference between real and fake images: $D(x) \rightarrow 1$ and $D(G(z)) \rightarrow 0$
- Generator trained to minimize it: $D(G(z)) \rightarrow 1$

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$

GAN – Generative Adversarial Networks

Trained using an iterative approach:

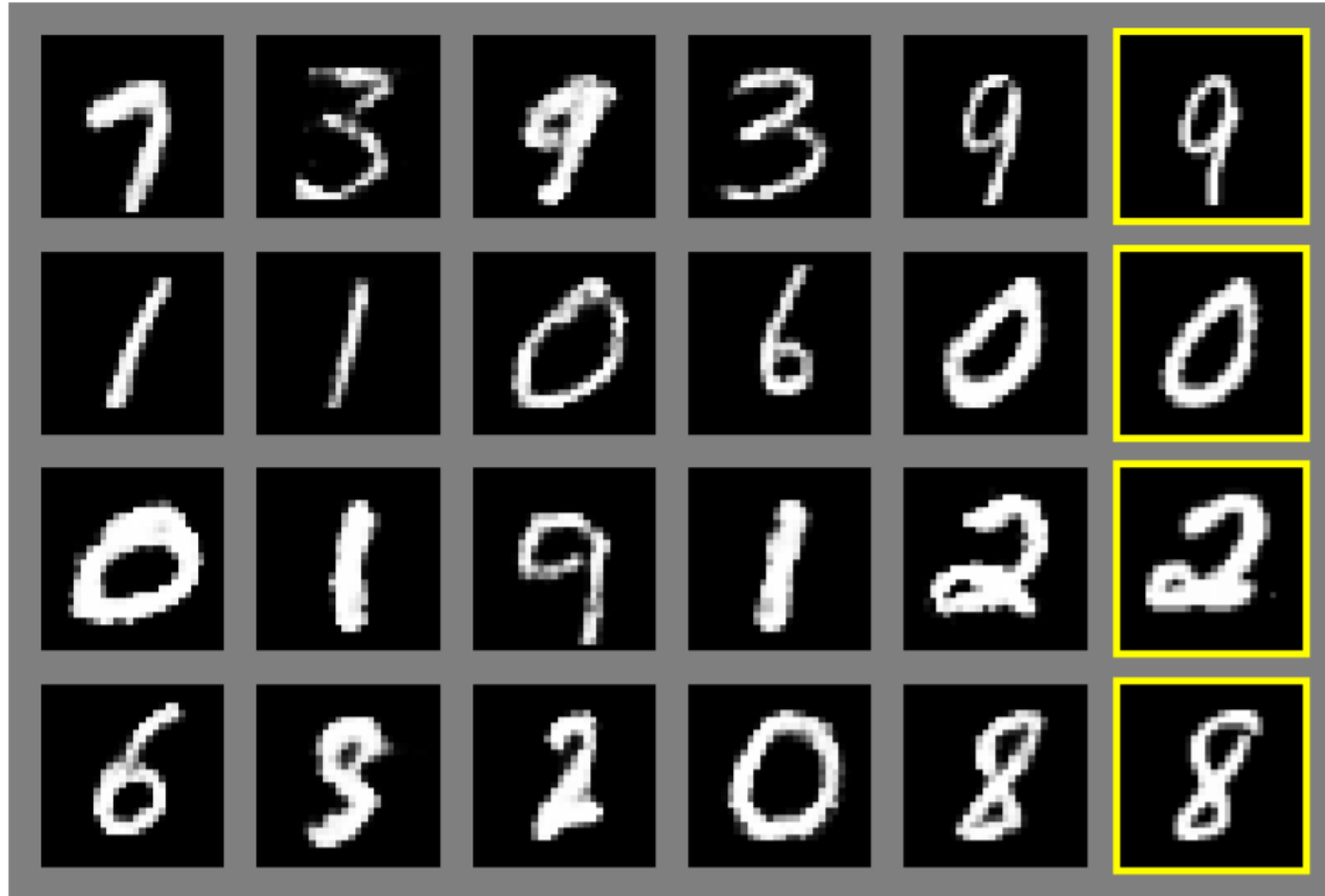
- Fix generator, train discriminator:

$$\max_D [\mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))]]$$

- Fix discriminator, train generator:

$$\min_G [\mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))]]$$

GAN – Generative Adversarial Networks



GAN – Generative Adversarial Networks



GAN – Generative Adversarial Networks

Training GANs is very difficult and unstable:

Not always converging

- Parameters are unstable and oscillate while training and never converge

Mode collapse

- Generator only produce same kind of images

Diminished gradient

- Strong discriminator causes small vanishing gradients for the generator

Very sensitive to choice of hyperparameters

StyleGAN

A Style-Based Generator Architecture for Generative Adversarial Networks

Tero Karras
NVIDIA

`tkarras@nvidia.com`

Samuli Laine
NVIDIA

`slaine@nvidia.com`

Timo Aila
NVIDIA

`taila@nvidia.com`

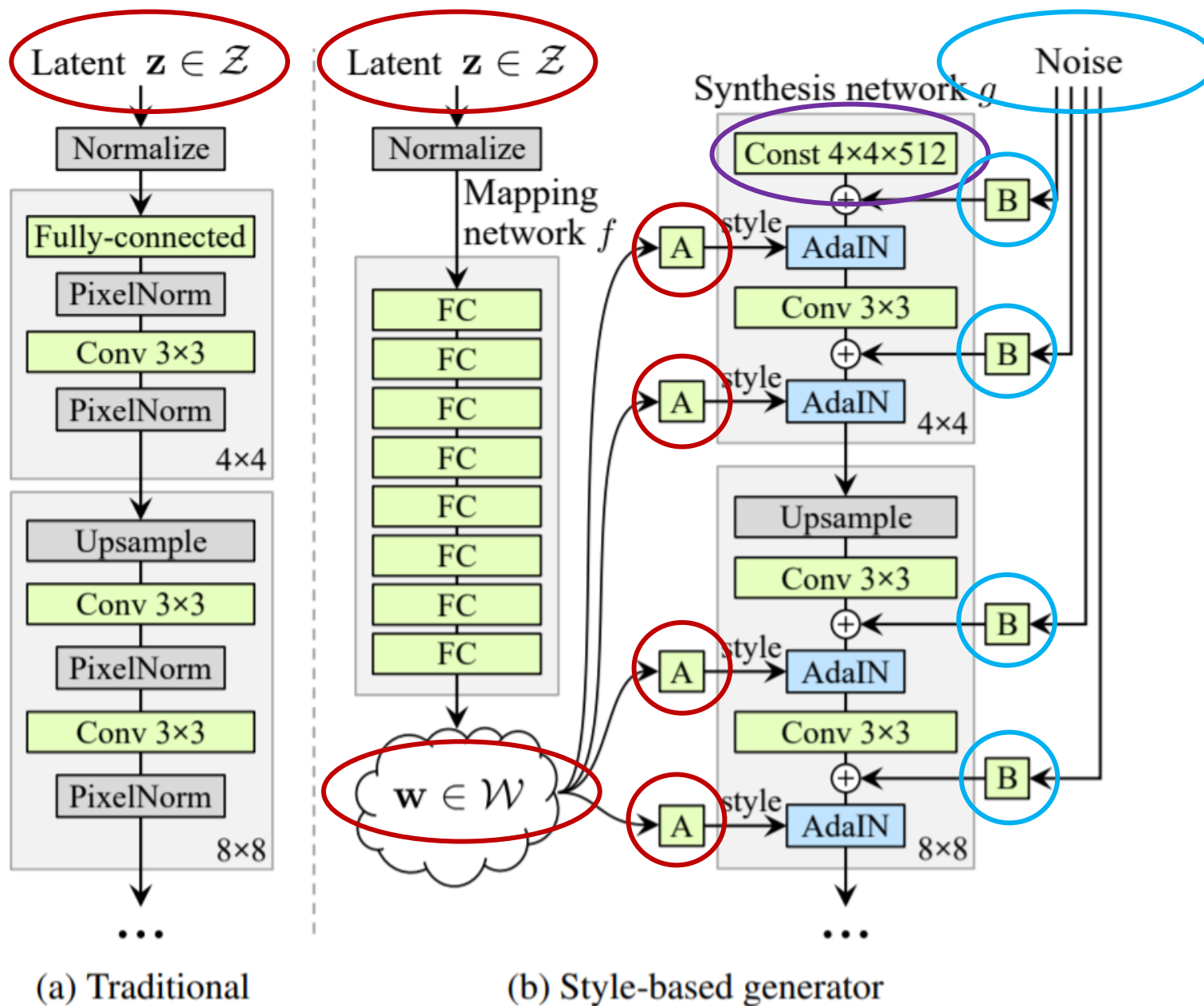
StyleGAN

We want better understanding of GANs and latent space

StyleGAN incorporates ideas from style transfer to separate high-level attributes (pose, identity, etc.) from stochastic variation (freckles, hair, etc.) in generated images

Only modifies the generator:

- Instead of using z as the input to the generator, we first map it to an intermediate latent space W using a non-linear mapping $f: Z \rightarrow W$.
- w will be inserted to the generator as a style “hint”
- Noise input is also given to the generator to help it generate stochastic detail



StyleGAN



StyleGAN



(a) Generated image (b) Stochastic variation

Generative Models

Unsupervised learning

Generate **new** samples from the same distribution of the training data

A generative model needs to be:

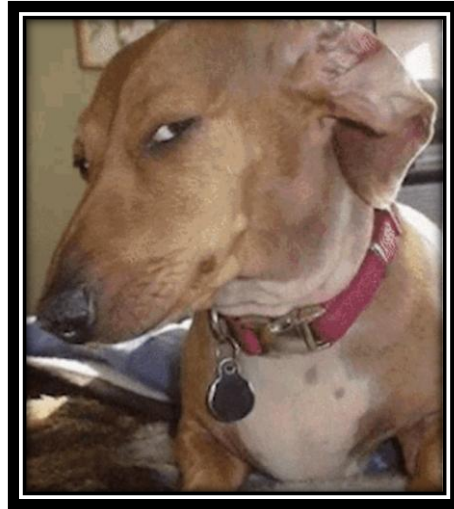
- Sufficient: for every image x there must be z s.t. $G(z) = x$
- Compact: for every z , $G(z)$ should be a valid image in X

Many ways to address this problem

- Auto-regressive pixel generation
- VAE – variational autoencoders
- GAN – generative adversarial networks
- **ViT – vision transformers**
- Diffusion models

What is CLIP?

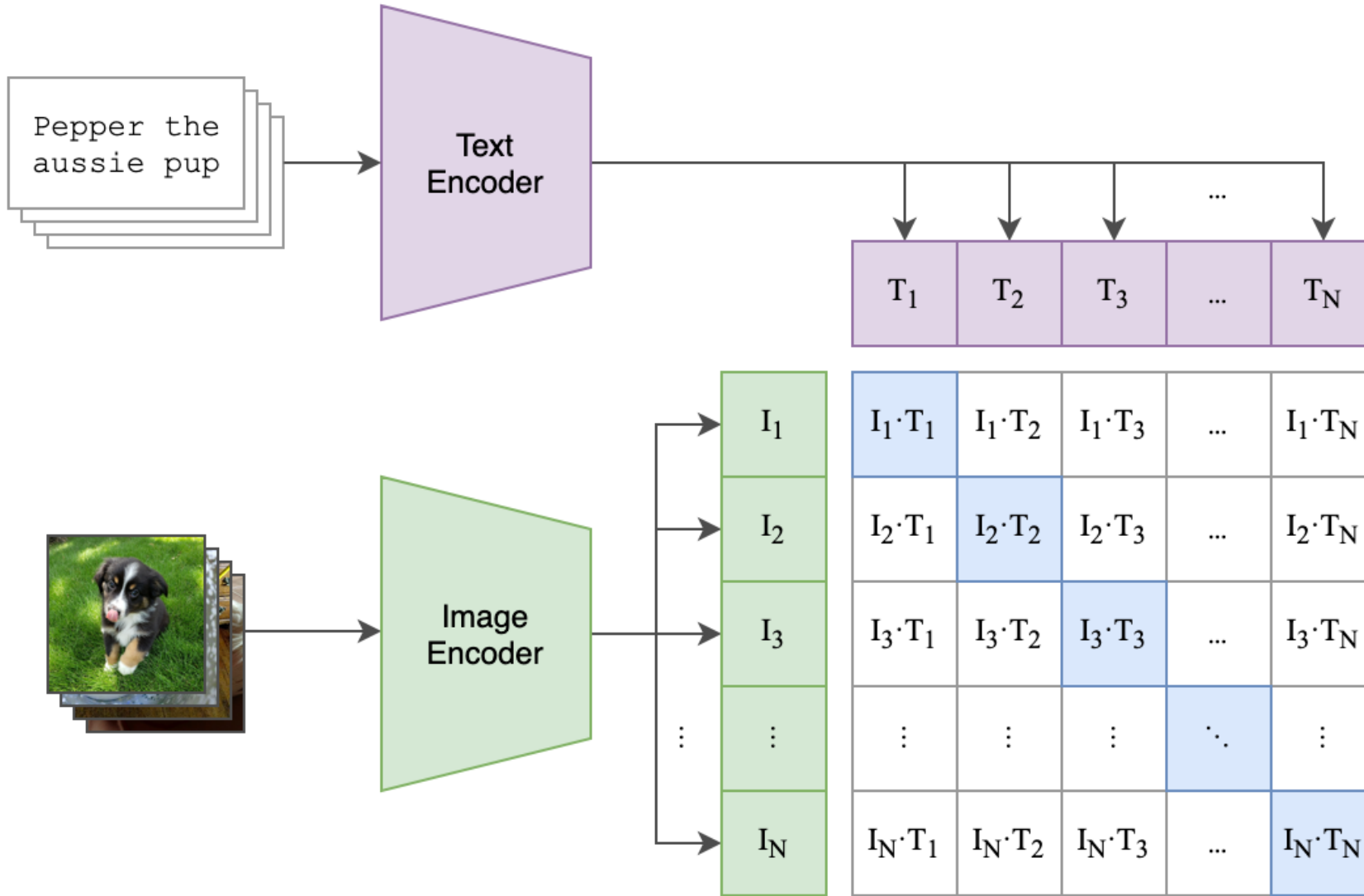
- CLIP = Contrastive Language-Image Pretraining
- **Goal:** Teaching a model to understand which text matches which image.



a photo of a dog

Instead of predicting labels, CLIP learns to **match the correct image with the correct text and separate mismatched pairs**

What is CLIP?



$$S_{i,j} = \text{similarity}(T_i, I_j)$$

$$S_{i,j} = \frac{T_i \cdot I_j}{\|T_i\| \|I_j\|}$$

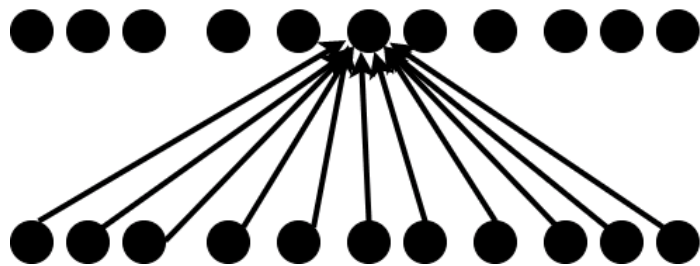
Attention Mechanism

Originally developed for NLP tasks.

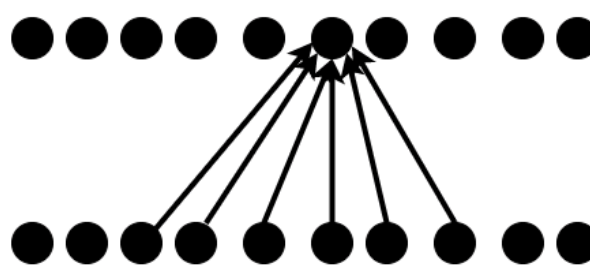
Not all words in a sentence are equally important - attention lets the model focus on relevant words.

Example: “The movie was **boring** but the acting was **amazing**.”

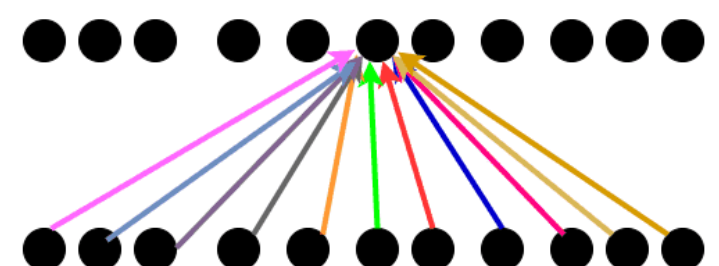
Fully Connected layer



Convolution

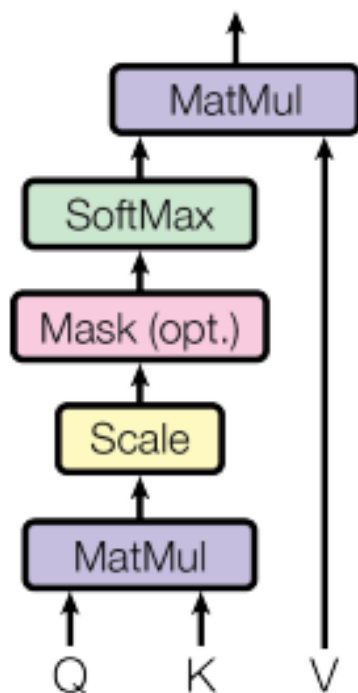


Global attention



How Attention Works (NLP)

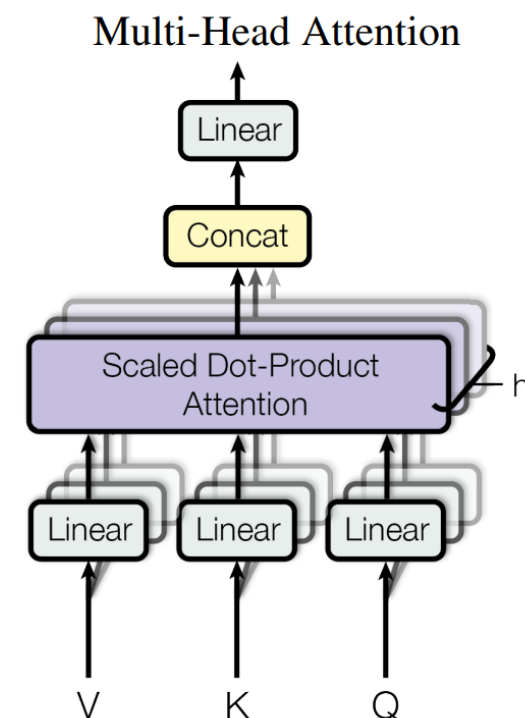
- Each word creates Query, Key, Value vectors.
- Attention scores measure how much one word relates to another.
- Weighted sum gives a context vector for each word.
- Enables the model to capture meaning across the whole sentence.



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

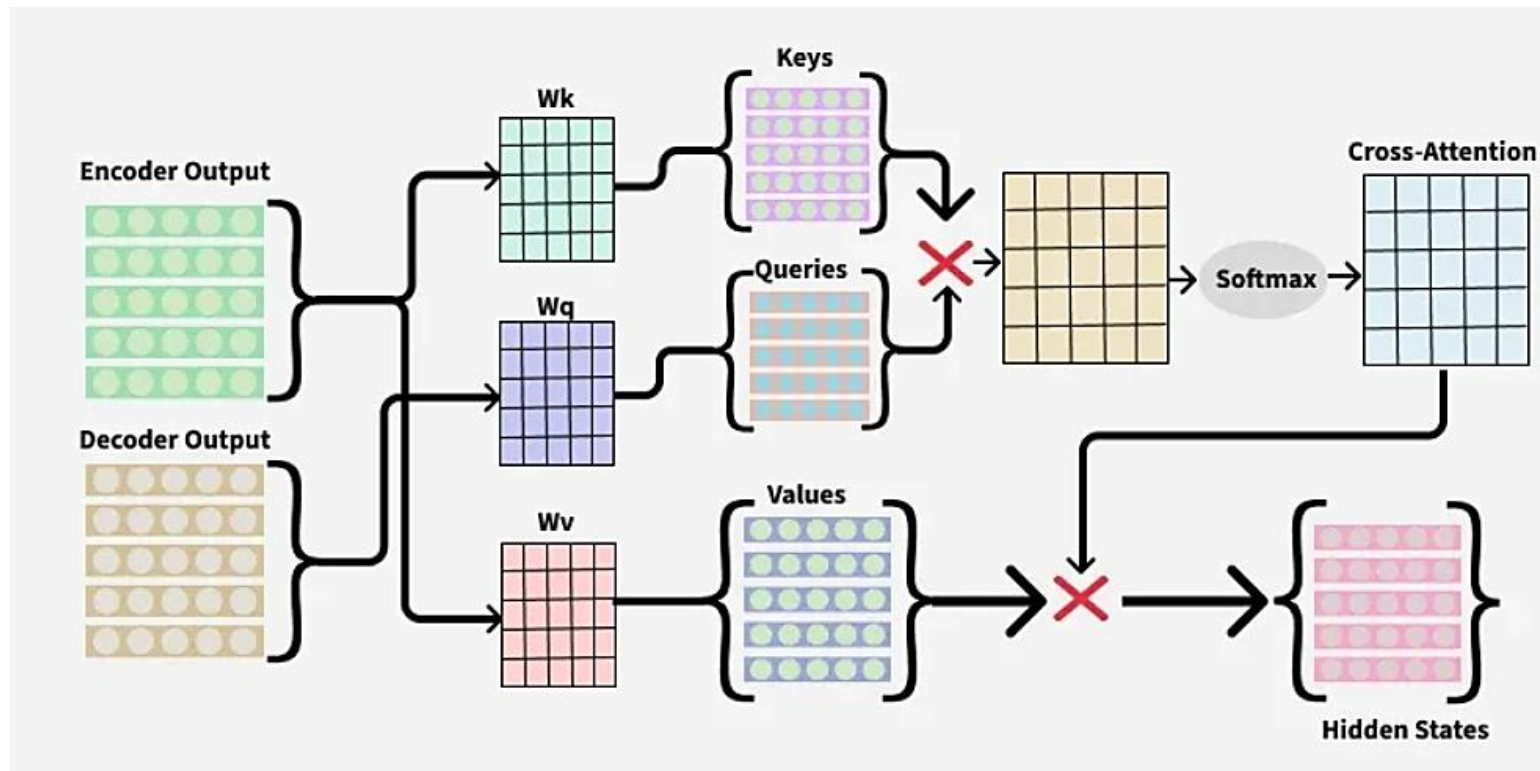
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



Cross Attention

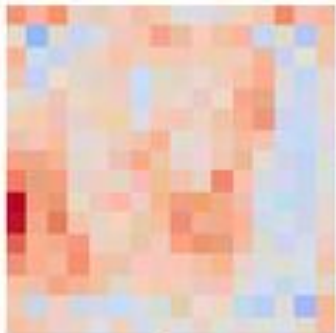
- Mechanism where one set of features attends to another set.
- Let's information flow between two different sources



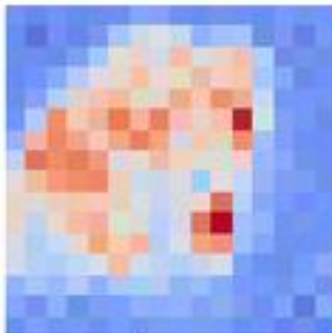
Cross Attention



Synthesized image



"a



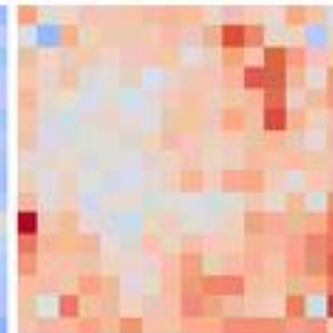
furry



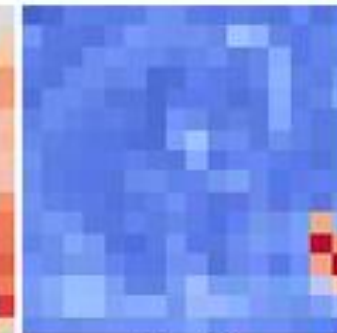
bear



watches

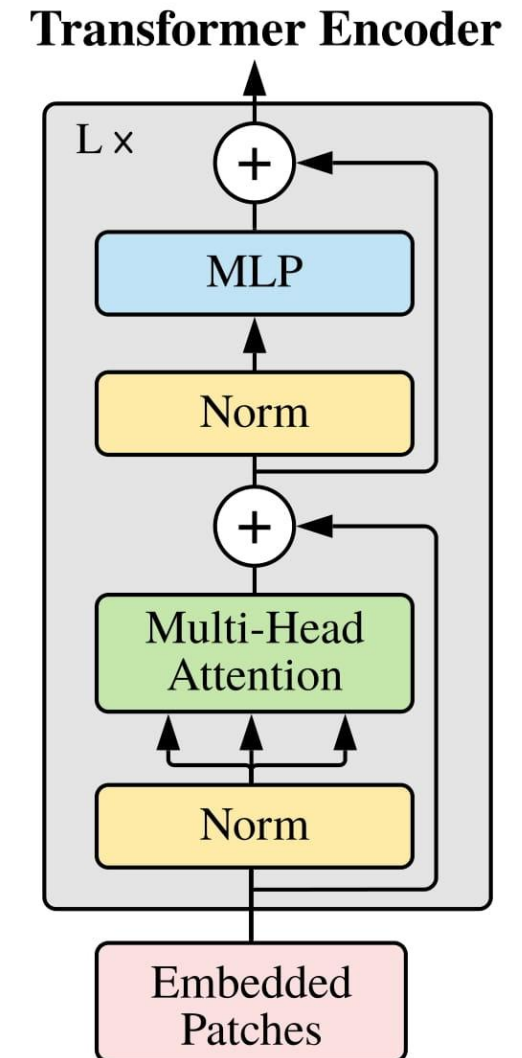
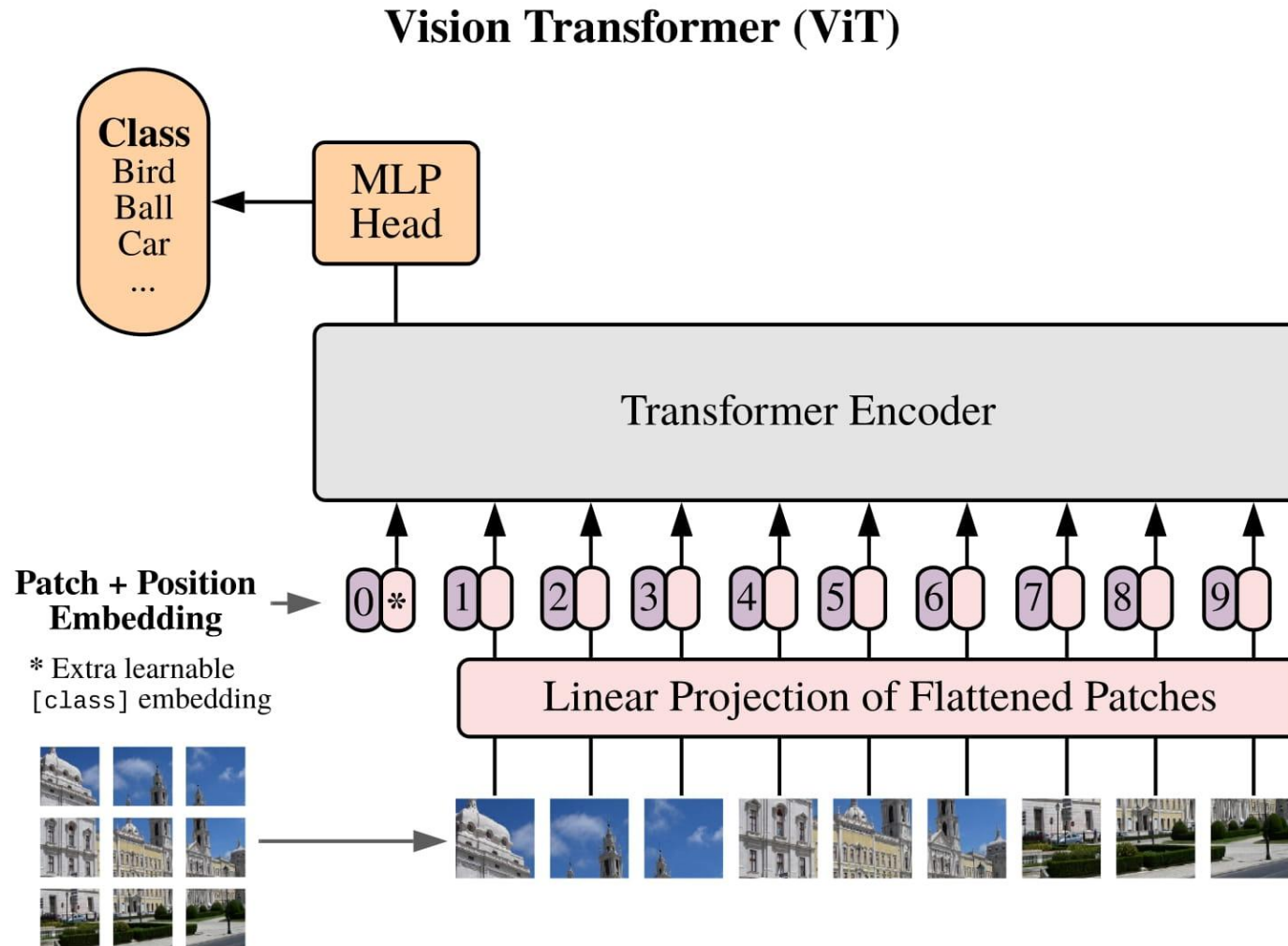


a



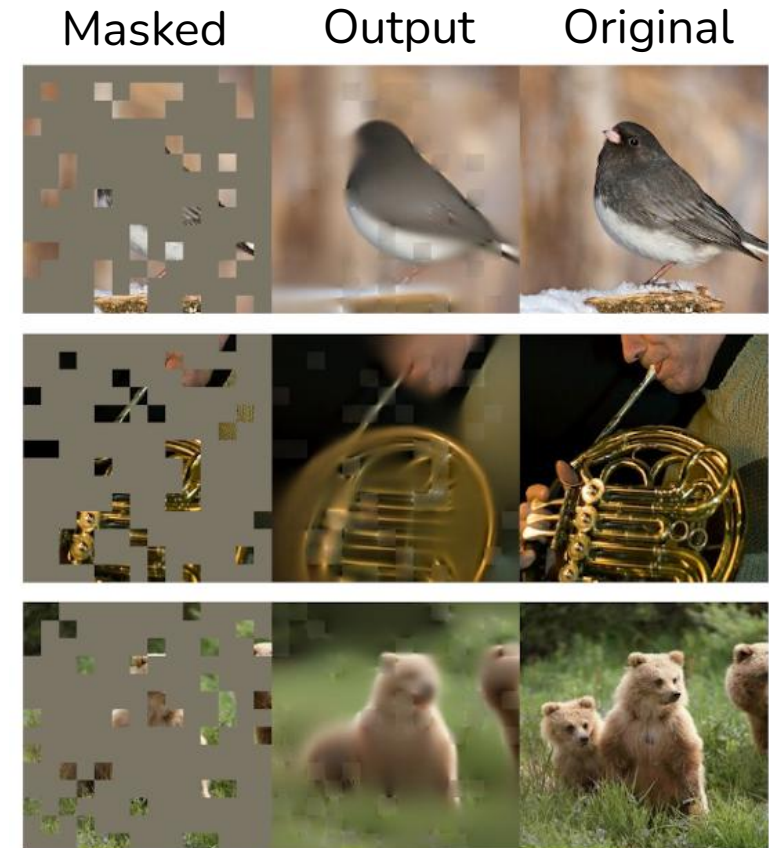
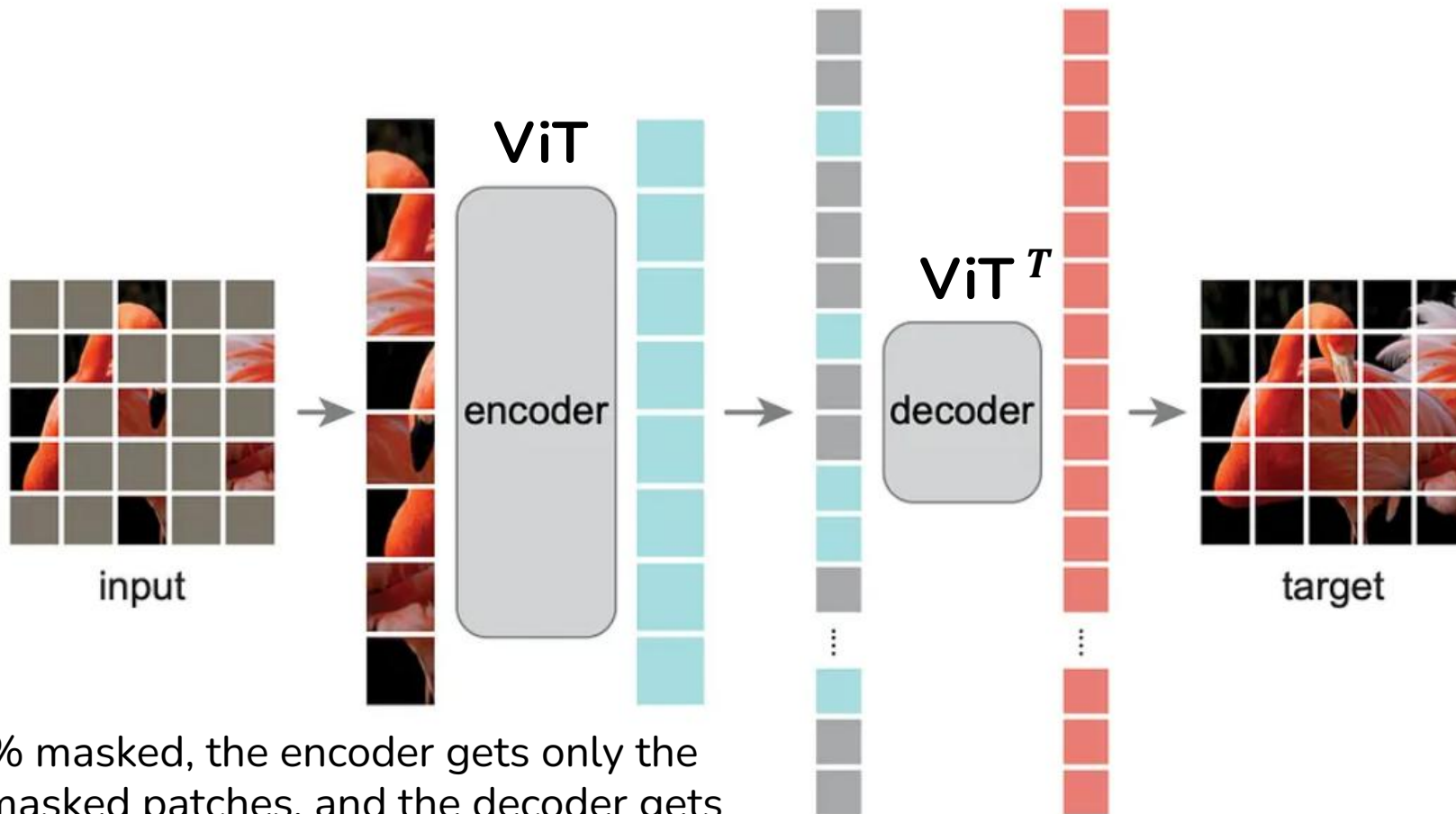
bird"

Vision Transformer (ViT)



Vision Transformer (ViT)

Masked Autoencoder (MAE)



75% masked, the encoder gets only the unmasked patches, and the decoder gets representations of masked where needed

Generative Models

Unsupervised learning

Generate **new** samples from the same distribution of the training data

A generative model needs to be:

- Sufficient: for every image x there must be z s.t. $G(z) = x$
- Compact: for every z , $G(z)$ should be a valid image in X

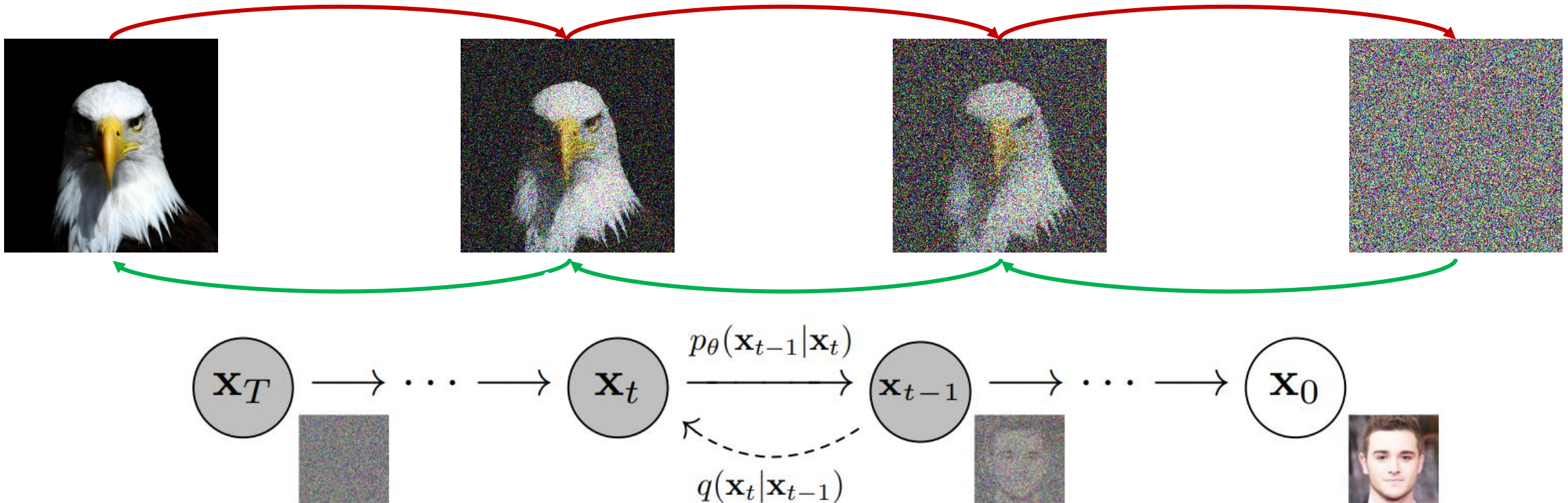
Many ways to address this problem

- Auto-regressive pixel generation
- VAE – variational autoencoders
- GAN – generative adversarial networks
- ViT – vision transformers
- **Diffusion models**

Denoising Diffusion Models

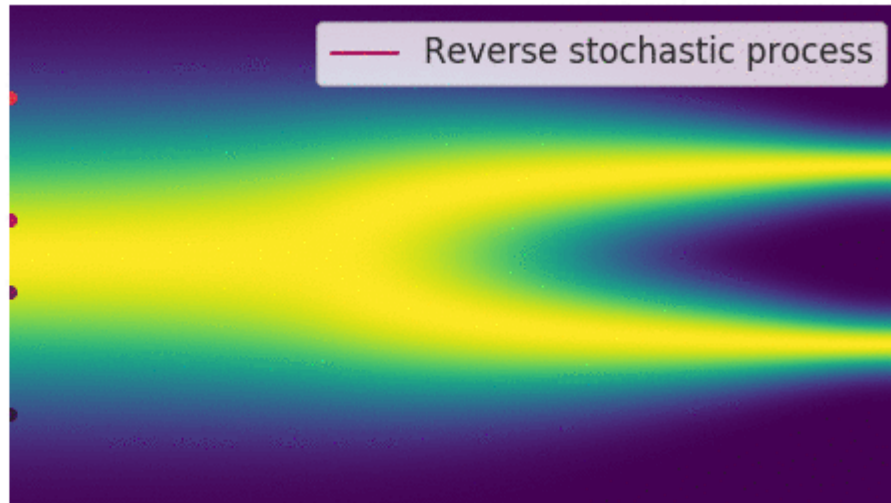
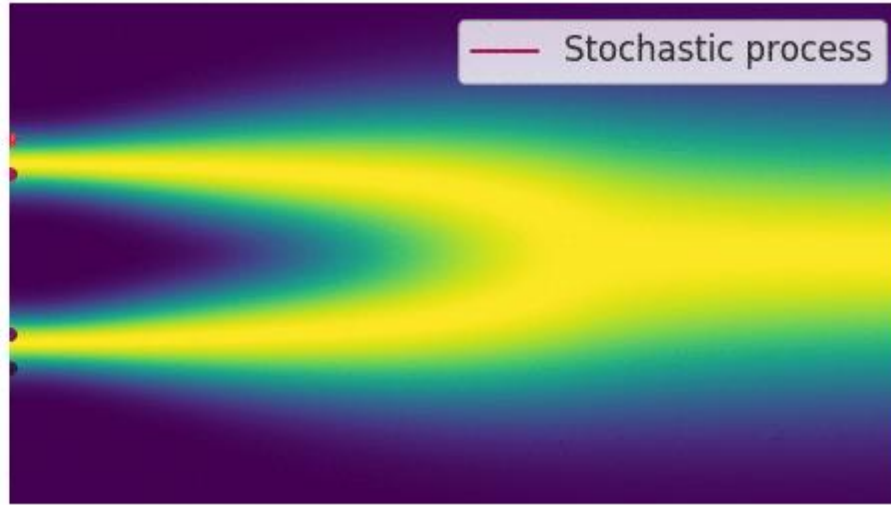
Diffusion models learn to invert a parameterized Markovian image noising process

- Can generate high-quality images



Denoising Diffusion Models

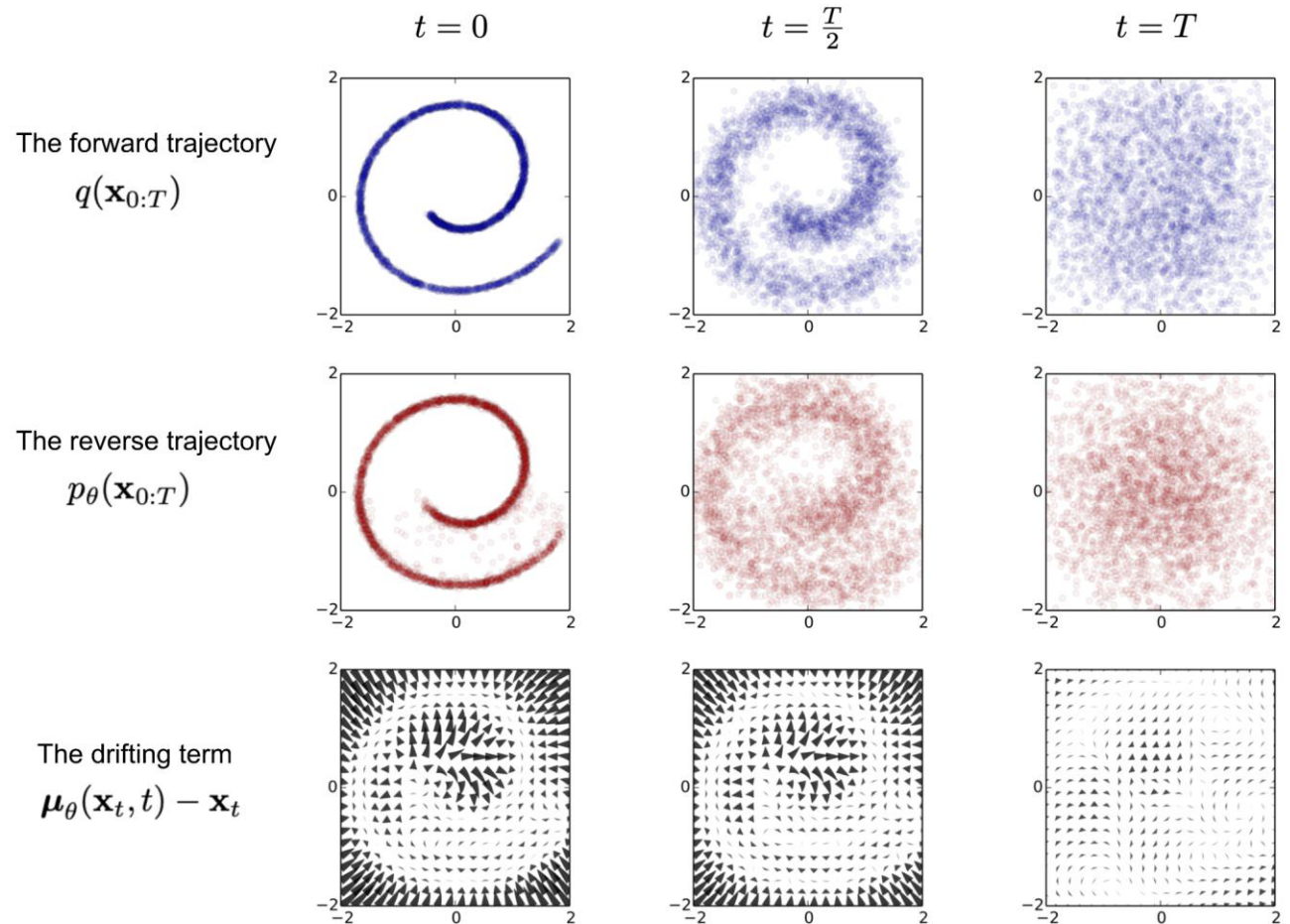
Forward &
Backward
Process



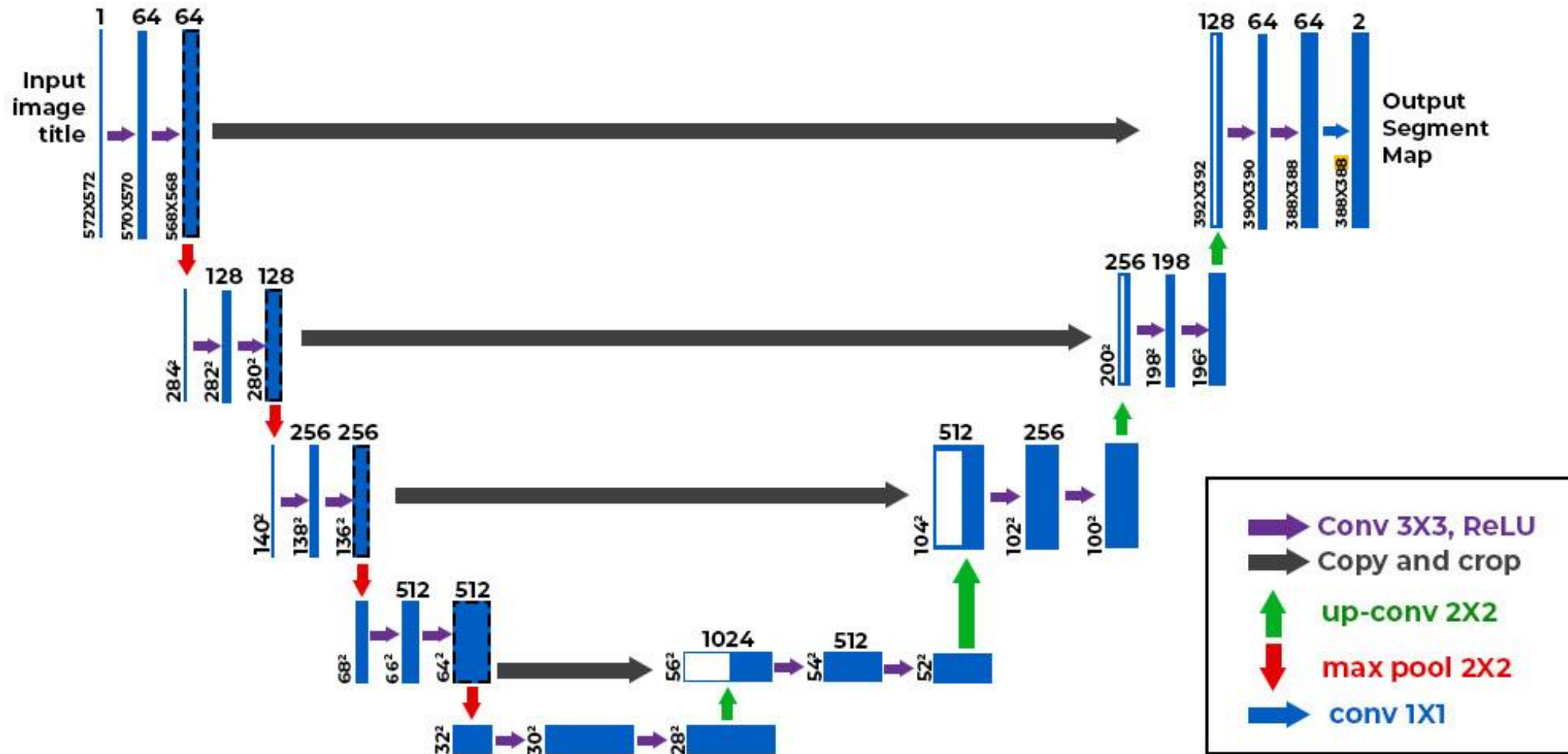
Denoising Diffusion Models

If you remember, in generative models we try to learn the probability distribution $p(x)$.

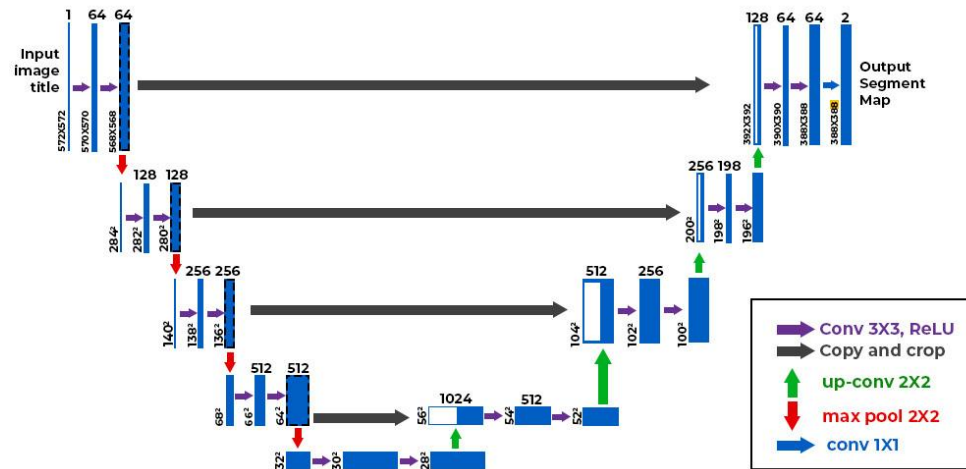
What we will learn is the “direction” each point should move to reach the true distribution.



Denoising Diffusion Models



Denoising Diffusion Models



Algorithm 1 Training

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Take gradient descent step on

$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$$
- 6: **until** converged

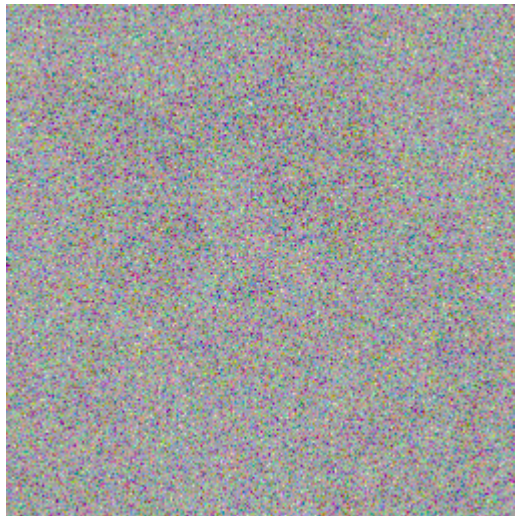
Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
- 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: **end for**
- 6: **return** $\mathbf{x}_0$

Denoising Diffusion Models

Two problems:

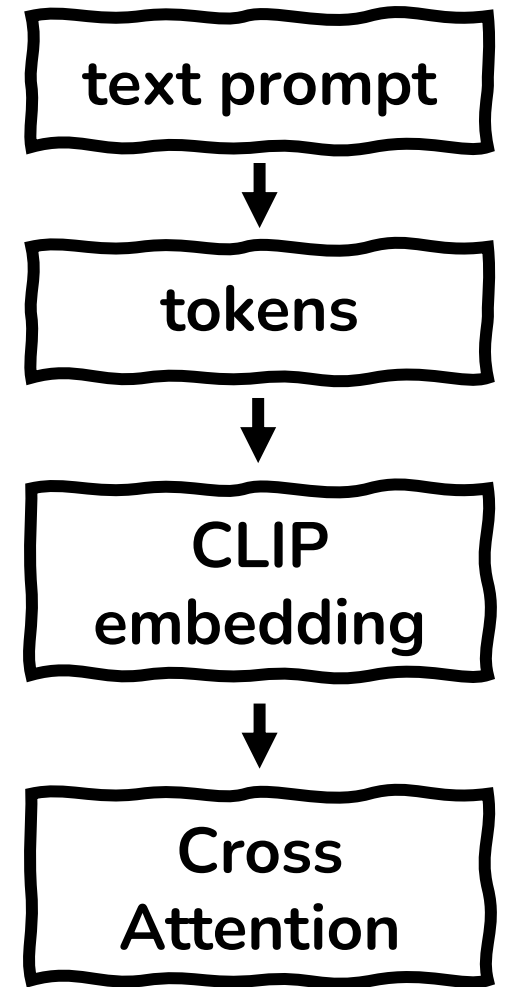
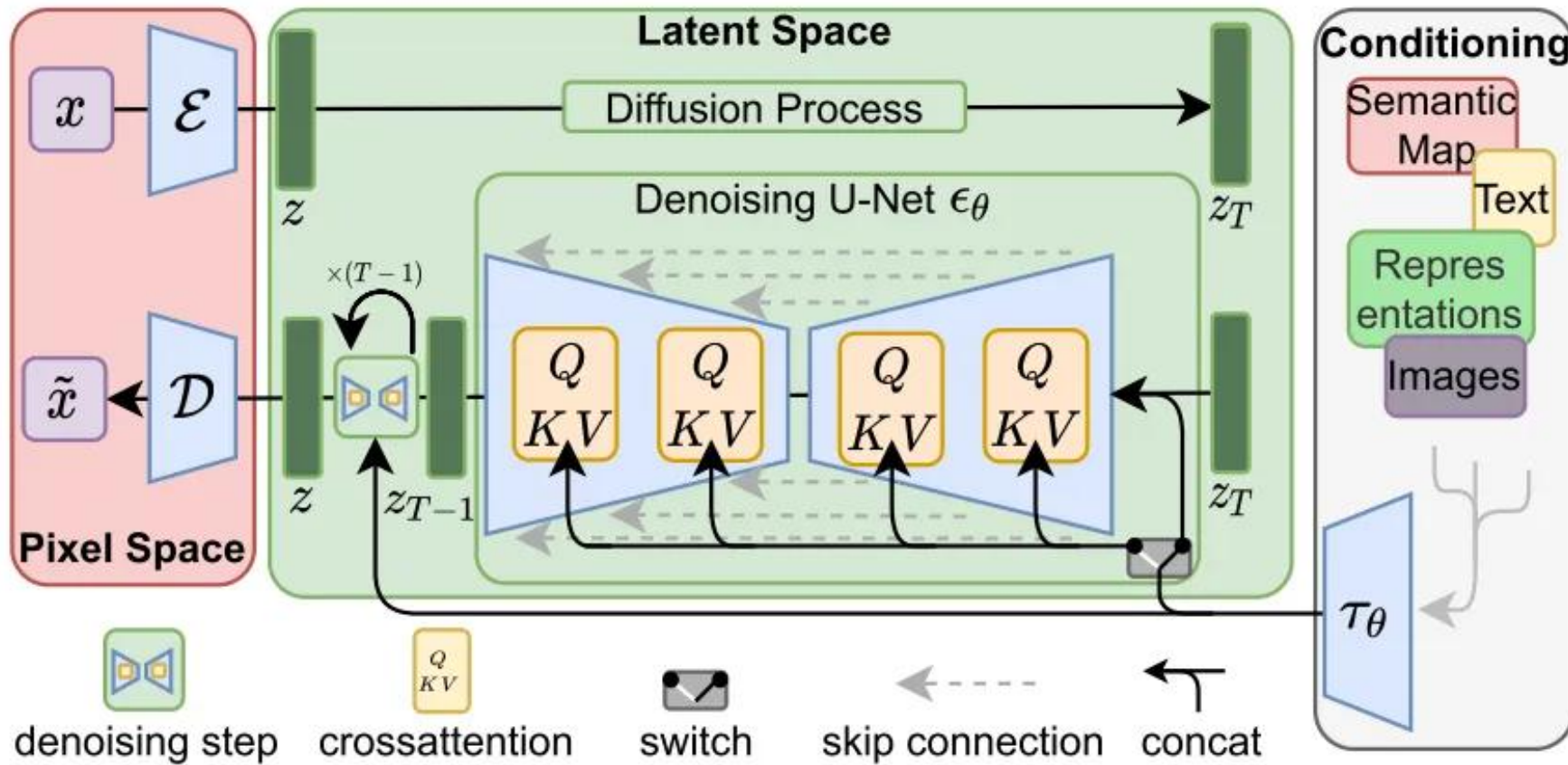
1. The model is unconditioned, so it generates a random image without any prompt or guidance.
2. For large images, the generation process takes a long time.



“But I’m allergic to cats!!!”

Denoising Diffusion Models

Solution:



Denoising Diffusion Models

DALL-E (OpenAI)

- “An astronaut riding a horse in a photorealistic style”
- “A bowl of soup that is a portal to another dimension as digital art”



Denoising Diffusion Models

Midjourney

- “hyperrealistic image of a man sitting on his bed watching his screen phone. he is smiling. he has short brown hair. it's evening.”
- “hyperrealistic image blue love letters aesthetic. top view. blue and black colors”



Denoising Diffusion Models

Sora

“a cute, tiny, squid balances on a human finger, in the background, it's the beach, with a beautiful sunset and beige sand, the water is blue and clear, but hyper realistic. the squid is absolutely adorable, with big eyes, and jelly-like skin, and it's a beautiful purple color. the squid is about the size of small hamster, and it's very cute. but the rest of the photo, including the jelly squid, is hyper realistic, but the squid looks like a tiny jelly.”



Next week:
Video Synopsis